TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 02

ỨNG DỤNG CỦA MẠCH SỐ

Họ và tên	Nguyễn Minh Đạt	**MSSV**	25520280
Lớp	IT012.R11.1	**Ngày thực hành**	05/10/2026
Tự chấm	12/10	**Link git**	https://github.com/25520280-debug/IT012.R11.1_25520280_Lab02.git

1. Tổng quan buổi thực hành

Sinh viên dùng phần mềm **Logisim** để biến kiến thức Chương 4 (mạch số, mạch tổ hợp, mạch tuần tự, latch, flip-flop, thanh ghi) và Chương 5 (bộ cộng, ALU, bộ chọn, bộ so sánh, bộ giải mã, tập thanh ghi) thành một bài toán xuyên suốt: **tự xây một datapath 8 bit của CPU thu nhỏ**, từng khối một, theo chu trình **dự đoán – dựng mạch – quan sát – giải thích**.

Sau buổi thực hành, Sinh viên có thể:

- Thiết kế và dựng bộ cộng/trừ từ cổng luận lý theo quy trình 4 bước (bảng chân trị → biểu thức → tối ưu → sơ đồ), đọc được các **cờ** Z, N, C, V và phát hiện **tràn số**.
- Dựng **ALU 8 bit** từ bộ chọn (MUX), bộ cộng/trừ và cổng luận lý; dùng ALU để so sánh hai số và chỉ ra **giới hạn** của quy tắc so sánh bằng phép trừ.
- Phân biệt **latch** (tích cực theo mức) với **flip-flop** (tích cực theo cạnh) bằng dạng sóng; tự dựng D latch và ghép hai latch thành D flip-flop.
- Giải thích vì sao thanh ghi dùng flip-flop và vì sao vòng hồi tiếp "thanh ghi → ALU → thanh ghi" vẫn hoạt động đúng.
- Dựng **tập thanh ghi 4 × 8 bit**, ghép với ALU thành đường dữ liệu và điều khiển bằng tay để chạy một chương trình ngắn (C = A + B, hoán đổi hai thanh ghi).

Phần	**Nội dung**	**Thời gian gợi ý**	**Điểm**
Phần 0	Làm quen bus nhiều bit, mạch con, clock; kiểm tra môi trường	15 phút	
TN 1	Bộ cộng/trừ 4 bit và các cờ trạng thái Z, N, C, V	45 phút	2,0
TN 2	ALU 8 bit và bộ so sánh: quy tắc nào luôn đúng?	45 phút	2,0
TN 3	Mạch tuần tự: D latch, D flip-flop	45 phút	3,5
TN 4	Tập thanh ghi và đường dữ liệu 8 bit (có phần mở rộng)	60 phút	4,0
Kết	Tổng kết, phản tư, nộp bài	15 phút	0,5
Tổng		**225 phút**	**12,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ Sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- Ký hiệu: NOT là dấu phẩy trên (A'); AND viết liền hoặc A·B; OR là A + B; XOR là A ⊕ B. Số thập lục phân viết 0x3C. Với bus nhiều bit, A[7] là bit trọng số cao nhất (MSB). Số có dấu dùng biểu diễn **bù 2** (Chương 2).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi mô phỏng; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.
- Các mạch con (subcircuit) tạo ở thí nghiệm trước được **dùng lại** ở thí nghiệm sau. Đặt tên đúng quy ước và không xoá.

PHẦN 0 — Làm quen bus, mạch con và clock (15 phút)

Sinh viên đã dùng Logisim ở Lab 01; phần này chỉ bổ sung các kỹ năng mới cho mạch nhiều bit và mạch tuần tự. Giảng viên hướng dẫn nhanh theo tài liệu *Hướng dẫn sử dụng Logisim*. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☑	Tạo project mới; đặt tên circuit đầu tiên là **Warmup**; lưu file **IT012_Lab02_<MSSV>.circ**	Mỗi thí nghiệm dùng các circuit riêng trong **cùng một file** (Project ▸ Add Circuit); tên circuit theo quy ước ở đầu mỗi thí nghiệm.
☑	Đặt Input Pin / Output Pin **nhiều bit** (thuộc tính Data Bits); đổi từng bit bằng Poke tool	Pin 8 bit hiển thị 8 ô bit. Dùng **Probe** (thư viện Wiring) và đổi thuộc tính **Radix** để đọc ở dạng nhị phân, thập lục phân, thập phân có dấu.
☑	Dùng **Splitter** để tách bus thành từng dây và ghép các dây thành bus	Fan Out = số dây, Bit Width In = độ rộng bus. Kiểm tra thứ tự bit (bit 0 là LSB).
☑	Tạo **mạch con**: dựng một circuit, rồi kéo circuit đó từ ngăn Explorer vào circuit khác như một linh kiện	Vị trí các pin trong circuit quyết định vị trí các chân của linh kiện; đặt nhãn pin rõ ràng.
☑	Dùng linh kiện có sẵn: **Plexers** (Multiplexer, Decoder), **Arithmetic** (Adder, Subtractor), **Memory** (D Flip-Flop, Register)	Chọn linh kiện rồi chỉnh Data Bits, Select Bits ở ngăn thuộc tính (góc dưới bên trái).
☑	Điều khiển mạch tuần tự: đổi tay một Input Pin **CLK**; dùng **Simulate ▸ Reset Simulation** để đưa mạch về trạng thái ban đầu	Cạnh lên xảy ra khi Poke CLK từ 0 lên 1. Với linh kiện Clock (Wiring), mỗi lần Simulate ▸ Tick Once đổi Clock một nửa chu kỳ.
☑	(Tuỳ chọn) Xem dạng sóng: **Chronogram** (Logisim-evolution) hoặc cửa sổ **Simulate ▸ Logging** (Logisim 2.7.x)	Tuỳ phiên bản; không bắt buộc, dùng để kiểm tra lại Bảng 10 ở TN 3.
☑	Chụp màn hình mạch và dán vào khung ảnh của file này	Chụp cả mạch lẫn nhãn ngõ vào/ngõ ra và giá trị đang hiển thị.

Phiên bản Logisim đang dùng	logisim-win-2.7.1

Đọc màu dây và giá trị trong Logisim

Xanh lá sáng = 1 · **Xanh lá đậm** = 0 · **Xanh dương** = chưa xác định / chưa được điều khiển · **Đỏ** = lỗi/xung đột · **Cam** = hai đầu dây lệch số bit.

Ở mạch **tuần tự**, ngay sau khi mở file hoặc Reset Simulation, giá trị đang lưu có thể **chưa xác định** và hiển thị là **x** (hoặc dây xanh dương) cho đến khi mạch được ghi lần đầu. Đó là hành vi bình thường, không phải lỗi; nếu phiên bản Logisim của Sinh viên hiển thị khác, hãy ghi lại điều quan sát được.

Bước 0 – Kiểm tra nhanh. Trong circuit **Warmup**, đặt hai Input Pin 8 bit **A**, **B**, một **Adder** (Arithmetic, Data Bits = 8), một Output Pin 8 bit **S** và một Probe. **Dự đoán** S (viết hex) bằng tay trước khi Poke, rồi đọc S ở dạng thập phân **không dấu** và **có dấu** (bù 2).

A	B	S = A + B (hex)	S không dấu	S có dấu (bù 2)
0x20	0x0F	0x2F	47	47
0x7F	0x01	0x80	128	-128
0xFF	0x01	0x00	0	0
0x64	0x64	0xC8	200	-56

Circuit Warmup chỉ để làm quen, không nộp. Hãy để ý: cùng một dãy bit, hai cách đọc cho hai giá trị khác nhau; Thí nghiệm 1 sẽ trả lời "phần cứng báo cho phần mềm biết kết quả có đáng tin hay không" bằng cách nào.

THÍ NGHIỆM 1 — Bộ cộng/trừ 4 bit và các cờ trạng thái

Thời lượng gợi ý: 45 phút · Circuit: **FA, ADDSUB4** · Điểm: 2,0

> **Tình huống**
> Phép cộng là phép toán nền tảng của ALU: phép trừ, tăng/giảm và so sánh đều quy về bộ cộng. Sinh viên sẽ tự xây bộ cộng 4 bit từ cổng luận lý, biến nó thành bộ cộng/trừ, rồi tự thiết kế mạch **phát hiện tràn số** để biết khi nào kết quả 4 bit không còn đúng, thông tin mà CPU thật luôn cung cấp dưới dạng các cờ.

A. Bộ cộng toàn phần (FA) từ cổng luận lý

Bước 1.1 – Dự đoán. Cộng ba bit A + B + Cin cho kết quả từ 0 đến 3, viết được bằng hai bit (Cout S). Điền hai cột "dự đoán" của Bảng 1 bằng phép cộng số học, **chưa** dựng mạch.

DỰNG MẠCH **Bước 1.2.** Trong circuit **FA** (circuit mới), đặt 3 Input Pin **A, B, Cin** và 2 Output Pin **S, Cout**. Dựng theo slide: S = A ⊕ B ⊕ Cin; Cout = A·B + Cin·(A ⊕ B), dùng chung cổng A ⊕ B cho cả hai ngõ ra.

QUAN SÁT Dùng Poke tool (hoặc Analyze Circuit ▸ Table) để điền hai cột "đo".

Bảng 1. Bộ cộng toàn phần FA

A	B	Cin	S (dự đoán)	Cout (dự đoán)	S (đo)	Cout (đo)
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	0	1	0	1
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	1

Câu 1.1 (Phân tích). Cột Cout chính là hàm "có từ 2 ngõ vào trở lên bằng 1" mà slide Chương 4 dùng làm ví dụ thiết kế. Viết SoP tối giản của Cout (gom bìa Karnaugh 3 biến nếu cần) và tính C(B). So sánh với dạng Cout = A·B + Cin·(A ⊕ B) khi cổng A ⊕ B đã có sẵn để tạo S: tổng số cổng của cả mạch FA ở mỗi cách là bao nhiêu?

C(B) = 3 + 2 + 2 + 2 = 9
Tổng số cổng của cả mạch FA theo slide : 6 cổng
Tổng số cổng của cả mạch FA theo đề bài : 5 cổng

Câu 1.2 (Phân tích). Đếm **số tầng cổng** (độ sâu) của mạch FA ở bước 1.2 trên hai đường đi: (a) từ **Cin** đến **Cout**; (b) từ **A hoặc B** đến **Cout**. Ghi cách đếm.

(a) Trả lời: (a) Cin→Cout: AND rồi OR = 2 tầng. (b) A/B→Cout: đường tạo P qua XOR, rồi AND, rồi OR = 3 tầng.
(b) : Từ A qua cổng XOR, qua cổng AND, qua cổng OR, rồi đến Cout là 3.

B. Ghép thành bộ cộng 4 bit nối tiếp

DỰNG MẠCH **Bước 1.3.** Trong circuit **ADDSUB4**: Input Pin **A** (4 bit), **B** (4 bit), **SUB** (1 bit); Output Pin **S** (4 bit), **C** (1 bit). **Lần này chưa dùng SUB:** nối 4 mạch con FA liên tiếp (Cout của tầng i vào Cin của tầng i + 1; Cin của tầng 0 nối hằng 0). Dùng Splitter để tách A, B và gộp S. Đặt Probe (hoặc Output Pin tạm) cho ba dây nhớ **C1, C2, C3** giữa các tầng.

DỰ ĐOÁN Với ba phép cộng ở Bảng 2, dự đoán S, các bit nhớ và **số tầng mà bit nhớ đã lan qua** (bắt đầu đếm từ khi C1 = 1).

Bảng 2. Bit nhớ lan truyền trong bộ cộng 4 bit (Cin = 0)

A	B	S (nhị phân)	C1	C2	C3	Cout	Số tầng bit nhớ lan qua
0001	0001	0010	1	0	0	0	1
0111	0001	1000	1	1	1	0	3
1111	0001	0000	1	1	1	1	4

Câu 1.3 (Đánh giá). Với bộ cộng nối tiếp n bit, bit nhớ có thể phải đi qua tất cả các tầng. Dựa vào Câu 1.2, điền số tầng cổng của đường đi dài nhất (từ A[0]/B[0] đến Cout) ở Bảng 3 và rút ra công thức theo n. Nhận xét: vì sao CPU 32/64 bit thực tế không dùng thuần bộ cộng nối tiếp?

Bảng 3. Độ sâu của bộ cộng nối tiếp

n (số bit)	Số tầng FA mà bit nhớ đi qua	Số tầng cổng đến Cout
4	4	9
8	8	17
32	32	65
n	n	2n + 1

Các CPU 32/64 bit thực tế thường không dùng thuần bộ cộng nối tiếp vì carry phải truyền qua nhiều tầng liên tiếp. Theo độ sâu dài nhất 2n+1, bộ cộng 64 bit có đường đi tới 129 tầng cổng, gây trễ lớn.

C. Biến thành bộ cộng/trừ

Câu 1.4 (Hiểu). A − B có thể viết là A + (−B), mà theo Chương 2, −B = B' + 1 (bù 2). Từ đó giải thích vì sao chỉ cần thêm 4 cổng XOR (mỗi cổng nhận B[i] và SUB) và nối SUB vào Cin của tầng 0 là bộ cộng thành bộ cộng/trừ. Điều gì xảy ra ở hai trường hợp SUB = 0 và SUB = 1?

B[i]⊕SUB tạo toán hạng B mới.
SUB=0: B không đổi, Cin=0 nên tính A+B.
SUB=1: B bị đảo thành B′, Cin=1 nên tính A+B′+1 = A−B theo bù 2.

DỰNG MẠCH **Bước 1.4.** Thêm vào ADDSUB4: 4 cổng XOR cho B'[i] = B[i] ⊕ SUB; nối SUB vào Cin của tầng 0. Kiểm tra nhanh: SUB = 0, A = 0011, B = 0100 → S = 0111; SUB = 1, A = 0111, B = 0011 → S = 0100.

D. Các cờ trạng thái và thiết kế mạch phát hiện tràn số

CPU thật không chỉ trả về kết quả S mà còn trả về các **cờ** để phần mềm biết kết quả có đáng tin hay không: **Z** (S = 0), **N** (S âm, tức S[3] = 1), **C** (bit nhớ ra Cout) và **V** (tràn số có dấu). Z, N, C có sẵn hoặc rất đơn giản; **V cần được thiết kế**, theo đúng quy trình 4 bước ở Chương 4.

THIẾT KẾ **Bước 1.5 – Lập bảng chân trị (bước 1).** V = 1 khi kết quả của phép cộng hai số bù 2 4 bit **không biểu diễn được** trong 4 bit (ngoài khoảng −8 … +7). Quan sát: hai số **cùng dấu** mà tổng **khác dấu** thì tràn; hai số **khác dấu** thì không bao giờ tràn. Đặt **X3** = A[3]; **Y3** = bit dấu của toán hạng thứ hai thực sự đi vào bộ cộng (= B[3] ⊕ SUB); **S3** = S[3]. Điền Bảng 4.

Bảng 4. Bảng chân trị của V(X3, Y3, S3)

X3	Y3	S3	V
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

THIẾT KẾ **Bước 1.6 – Biểu thức và tối ưu (bước 2 và 3).** Viết SoP chính tắc của V, điền bìa Karnaugh, thử gom nhóm, rồi tính chi phí C(B).

X3 \ Y3 S3	00	01	11	10
0	0	1	0	0
1	0	0	0	1

SoP chính tắc của V:	X3'Y3'S3 + X3Y3S3'
Có gom được nhóm 2 ô không? Vì sao?	Không, do hai ô không kề nhau.
V tối giản = … ; C(B) = …	SoP tối giản= X3'Y3'S3 + X3Y3S3' C(B)=3+3+2=8.

Câu 1.5 (Phân tích). Biểu thức V có thể viết gọn là V = (X3 ⊕ S3)·(Y3 ⊕ S3). Chứng minh dạng này bằng Bảng 4 (hai thừa số cùng bằng 1 khi nào?) rồi so sánh số cổng với SoP ở Bước 1.6. Vì sao bìa Karnaugh "bó tay" với hàm này (liên hệ hàm chẵn lẻ/XOR ở Lab 01)?

(X⊕S) và (Y⊕S) cùng bằng 1 khi X≠S và Y≠S, tức X=Y nhưng S trái dấu có đúng hai trường hợp tràn ở Bảng 4, Bìa Karnaugh không gom nhóm được vì hai minterm 001 và 110 không kề nhau.
Dạng XOR dùng 2 XOR + 1 AND = 3 cổng, ít hơn SoP (2 AND3 + OR2 và thêm 3 cổng NOT).

DỰNG MẠCH **Bước 1.7.** Thêm vào ADDSUB4 các ngõ ra: **Z** = NOR của 4 bit S (slide Chương 5: "A == 0"), **N** = S[3], **C** = Cout, **V** theo biểu thức của Sinh viên (chọn dạng ít cổng hơn).

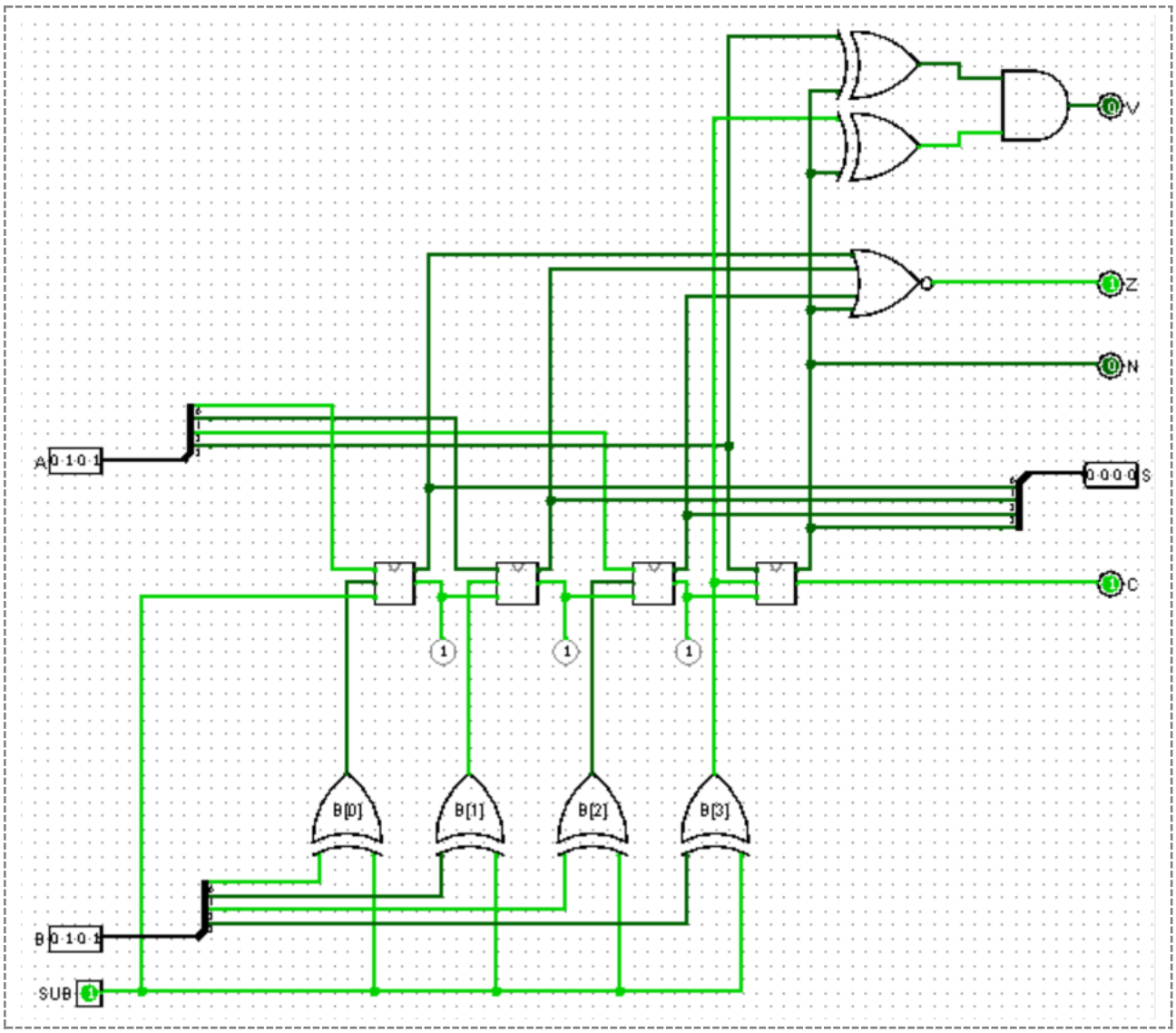
QUAN SÁT Đặt A, B, SUB theo Bảng 5, ghi S, Z, N, C, V. Sau đó **tự tính bằng toán học** xem S có còn đúng khi hiểu A, B là số **có dấu** (−8 … 7) hoặc số **không dấu** (0 … 15) hay không (điền Đ hoặc S).

Bảng 5. Kiểm thử các cờ của ADDSUB4

A	B	Phép	S	Z	N	C	V	Đúng nếu hiểu là số có dấu?	Đúng nếu hiểu là số không dấu?
0101	0011	A + B	1000	0	1	0	1	S	Đ
1111	0001	A + B	0000	1	0	1	0	Đ	Đ
1000	1000	A + B	0000	1	0	1	1	Đ	Đ
0011	0111	A − B	1100	0	1	0	0	Đ	S
0111	1111	A − B	1000	0	1	0	1	S	Đ
0101	0101	A − B	0000	1	0	1	0	Đ	Đ

Câu 1.6 (Phân tích). (a) Từ Bảng 5, chỉ ra dòng có (C, V) = (0,1), (1,0), (1,1), (0,0). Có thể dùng C thay cho V (hoặc ngược lại) không? Mỗi cờ báo "kết quả sai" cho cách hiểu nào (có dấu / không dấu)? (b) Ở các dòng phép **trừ**, C = 1 khi nào? Phát biểu điều kiện đó theo A và B không dấu.

(C,V) bằng các giá trị sau khi :
(0,1) khi 0101+0011
(1,0) khi 1111+0001 và 0101−0101
(1,1) khi 1000+1000
(0,0) khi 0011-0111 và 0111-1111
Không thể thay C cho V: C báo carry cho số không dấu, V báo tràn số có dấu.
Khi trừ A−B, C=1 khi không có nhớ, tương đương A >= B nếu hiểu A,B là số không dấu.


V
Z
N
A
0 1 0 1
0 0 0 0
S
C
1
1
1
B[0]
B[1]
B[2]
B[3]
B
0 1 0 1
SUB
1

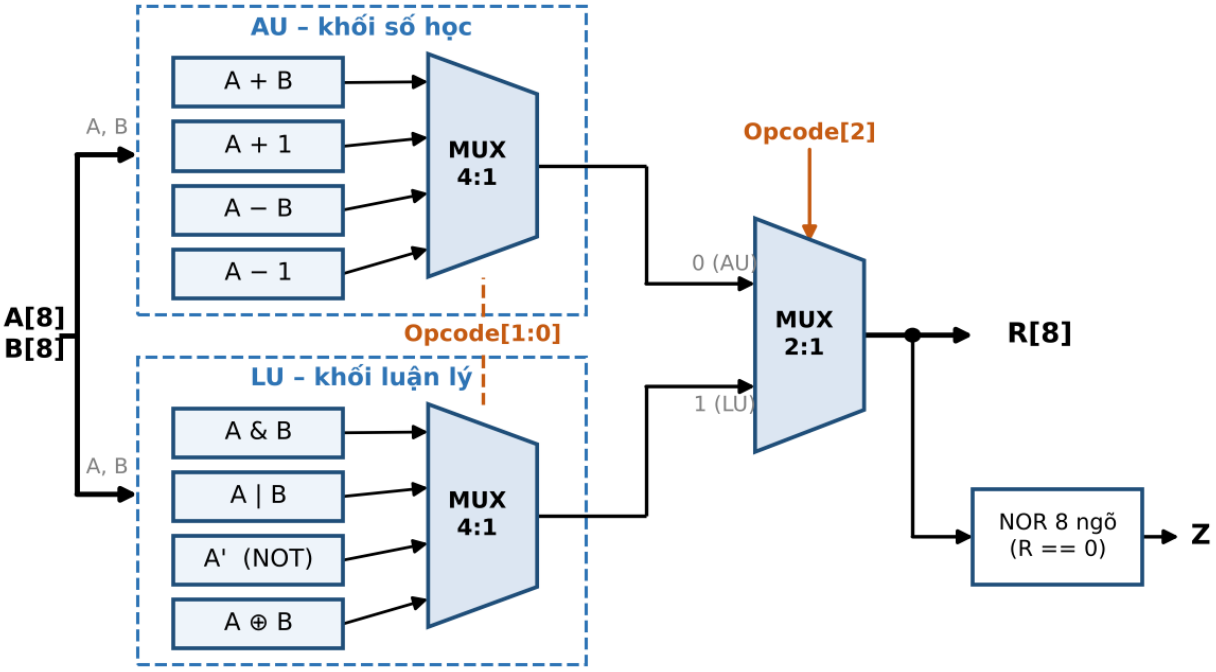
THÍ NGHIỆM 2 — ALU 8 bit và bộ so sánh: quy tắc nào luôn đúng?

Thời lượng gợi ý: 45 phút · Circuit: **ALU8, LT** · Điểm: 2,0

Tình huống

Một CPU chỉ cần **một** khối tính toán dùng chung cho nhiều phép toán: ALU. Sinh viên dựng ALU 8 bit theo bảng Opcode ở slide Chương 5 (Hình 1), rồi dùng chính ALU để **so sánh hai số** thay vì thiết kế bộ so sánh riêng, vì bảng chân trị của bộ so sánh hai số 8 bit có 2^16 hàng. Slide nêu quy tắc: "Y = A − B; Y[MSB] = 1 ⇒ A < B; Y[MSB] = 0 ⇒ A > B". Sinh viên sẽ **kiểm tra** quy tắc này có luôn đúng không.

A. Dựng ALU8 theo cấu trúc AU – LU – MUX



Hình 1. *Cấu trúc ALU 8 bit theo slide Chương 5. Opcode[1:0] chọn phép toán trong từng khối; Opcode[2] chọn khối AU hay LU.*

DỰNG MẠCH **Bước 2.1.** Trong circuit **ALU8**: Input Pin **A, B** (8 bit), **OP** (3 bit); Output Pin **R** (8 bit), **Z**. (a) **AU:** hai Adder (A + B, A + 1) và hai Subtractor (A − B, A − 1); hằng 1 lấy từ Constant 8 bit; chọn kết quả bằng Multiplexer 4:1 (8 bit) với Select = OP[1:0] (dùng Splitter tách OP). (b) **LU:** các cổng AND, OR, NOT, XOR có Data Bits = 8; chọn bằng một Multiplexer 4:1 khác với Select = OP[1:0]. (c) Multiplexer 2:1 (8 bit) chọn giữa AU và LU theo OP[2]. (d) **Z** = NOR của 8 bit R (Splitter + cổng NOR 8 ngõ vào).

Gợi ý: ngõ vào số k của Multiplexer được chọn khi Select = k. Hãy đối chiếu thứ tự nối với bảng Opcode ở slide (000: A + B, 001: A + 1, 010: A − B, 011: A − 1, 100: A & B, 101: A | B, 110: A', 111: A ⊕ B).

DỰ ĐOÁN Điền cột "dự đoán" của Bảng 6 bằng tay (hex, 8 bit) **trước khi** Poke.

QUAN SÁT **Ca 1:** A = 0x3C, B = 0x0F. **Ca 2:** A = 0x00, B = 0x01. Điền cột "đo" và cờ Z của Ca 2.

Bảng 6. Kiểm thử ALU8 với 8 Opcode

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
000	A + B	0x4B	0x4B	0x01	0x01	0

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
001	A + 1	0x3D	0x3D	0x01	0x01	0
010	A − B	0x2D	0x2D	0xFF	0xFF	0
011	A − 1	0x3B	0x3B	0xFF	0xFF	0
100	A & B	0x0C	0x0C	0x00	0x00	1
101	A | B	0x3F	0x3F	0x01	0x01	0
110	A'	0xC3	0xC3	0xFF	0xFF	0
111	A ⊕ B	0x33	0x33	0x01	0x01	0

Câu 2.1 (Phân tích). Ở Ca 2, phép nào cho R = 0xFF? Giá trị đó là 255 hay −1? ALU có "biết" điều đó không? Z = 1 ở phép nào, và Z = 1 có đồng nghĩa với "hai toán hạng bằng nhau" không?

Ở Ca 2, R=0xFF tại A-B (010), A−1 (011), A′ (110).
Cùng mẫu bit 11111111 có thể đọc là 255 không dấu hoặc −1 có dấu.
ALU chỉ tạo bit, không biết cách diễn giải.
Z=1 chỉ ở A&B (100), vì kết quả phép đó bằng 0.
Z=1 tức kết quả hiện tại bằng 0, không có nghĩa là hai toán hạng bằng nhau.

Câu 2.2 (Áp dụng). ALU này không có phép "chép" (MOV: R = A). Đề xuất **hai** cách dùng đúng ALU này để có R = A (ghi Opcode và giá trị cần đặt cho B). Cách nào dùng được khi ta **chỉ chọn được thanh ghi** cho A và B, không đặt được hằng số? (Sẽ dùng ở Thí nghiệm 4.)

Cách 1: OP=101 (OR), B=0 ⇒ A|0=A.
Cách 2: OP=101, B=A (chọn cùng thanh ghi cho A và B) ⇒ A|A=A.
Cách 2 dùng được khi chỉ chọn thanh ghi, không nạp hằng.
Ngoài ra XOR với B=0 (OP=111) cũng cho A nếu có thanh ghi 0.

B. So sánh hai số bằng ALU: kiểm tra quy tắc ở slide

DỰ ĐOÁN Với 6 cặp (A, B) ở Bảng 7, tính R = A − B trên **8 bit**, rồi điền cột "Quy tắc slide kết luận" (dùng <, =, >). Điền hai cột "Thực tế" theo hai cách hiểu: A, B là số **có dấu** (bù 2) và là số **không dấu**.

QUAN SÁT Đặt Opcode = 010 trên ALU8, ghi R, Z, R[7] của từng cặp. Cột **LT đo** để trống, sẽ điền sau Bước 2.3.

Bảng 7. So sánh bằng phép trừ

A	B	R = A − B (đo)	Z	R[7]	Quy tắc slide kết luận	Thực tế (có dấu)	Thực tế (không dấu)	LT đo
0x05	0x07	0xFE	0	1	<	<	<	1
0x09	0x09	0x00	1	0	>	=	=	0
0x64	0x9C	0xC8	0	1	<	>	<	0
0x9C	0x64	0x38	0	0	>	<	>	1
0xC8	0x0A	0xBE	0	1	<	>	<	1
0x0A	0xC8	0x42	0	0	>	>	<	0

Câu 2.3 (Phân tích). Tìm các dòng mà quy tắc slide cho kết luận **sai**. Với mỗi dòng sai, nêu nguyên nhân (gợi ý: phép trừ 8 bit có bị tràn không? A và B đang được hiểu là số có dấu hay không dấu?). Quy tắc ở slide chỉ đáng tin trong điều kiện nào?

Sai ở cách hiểu có dấu với 0x64-0x9C và 0x9C-0x64: phép trừ tràn có dấu nên MSB của kết quả không còn cho dấu thật.
Sai ở cách hiểu không dấu với 0xC8−0x0A và 0x0A-0xC8: MSB không phải cờ so sánh không dấu.
Dòng bằng nhau 0x09-0x09 cho 0, không phải >. MSB chỉ có thể chỉ ra A<B có dấu khi không tràn; muốn kết luận đúng cần xét Z để phân biệt bằng nhau và V để hiệu chỉnh dấu.
Với không dấu, dùng C (A<B khi C=0).

THIẾT KẾ **Bước 2.2 – Sửa quy tắc cho số có dấu.** Thiết kế mạch **LT** (A < B, số có dấu) chỉ dựa vào ba bit A[7], B[7], R[7] (R = A − B). Gợi ý: nếu A và B cùng dấu thì phép trừ không bao giờ tràn nên LT = R[7]; nếu khác dấu thì LT phụ thuộc dấu của A. Điền Bảng 8, bìa Karnaugh, rồi viết biểu thức tối giản.

Bảng 8. Bảng chân trị của LT(A7, B7, R7)

A7	B7	R7	LT (A < B có dấu)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

A7 \ B7 R7	00	01	11	10
0	0	1	0	0
1	1	1	1	0

LT tối giản = …	A7.B7’ + A7.B7 + B7’.R7
Số cổng cần dùng (kể cả cổng NOT)	5

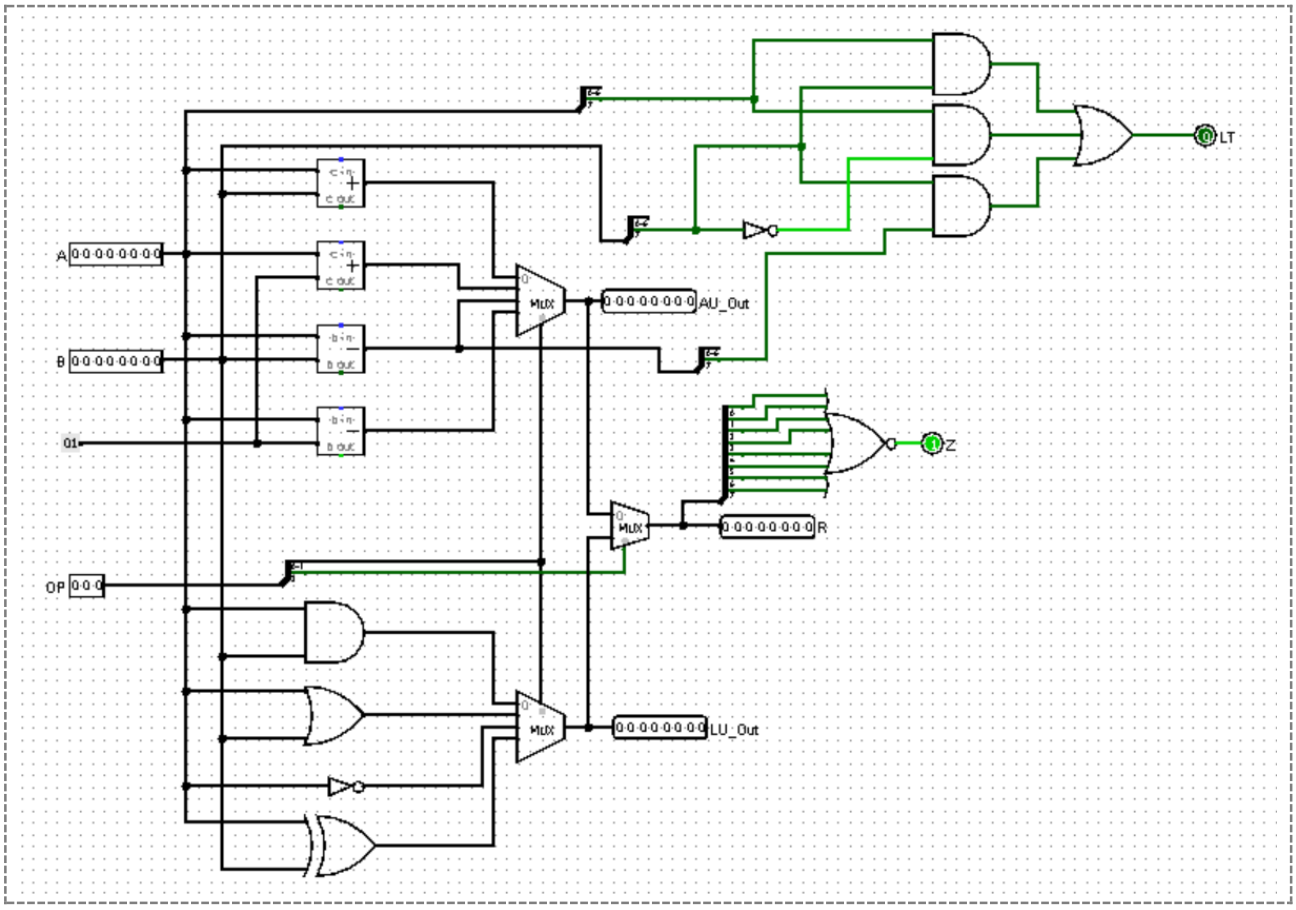
Câu 2.4 (Phân tích). Biểu thức LT quen thuộc không? (Gợi ý: so sánh với hàm đa số ở slide Chương 4 và với Cout của FA ở Thí nghiệm 1, trong đó một ngõ vào được lấy bù.) Giải thích bằng lời vì sao có sự giống nhau đó (nhớ lại A − B = A + B' + 1).

LT = A7.B7’ + A7.B7 + B7’.R7 giống Cout của FA, vì phép trừ A-B=A+B’+1 tạo carry.
Nguyên nhân có sự giống nhau đó đến từ việc cả 2 đều dùng phép trừ số có dấu trong máy tính được thực hiện thông qua phép cộng bù 2

DỰNG MẠCH **Bước 2.3.** Dựng mạch con **LT** (ngõ vào A7, B7, R7; ngõ ra LT), nối vào ALU8 (A[7], B[7], R[7]) và điền cột **LT đo** của Bảng 7. LT có sửa được các dòng sai ở Câu 2.3 không?

Câu 2.5 (Đánh giá – mở rộng). Với số **không dấu**, A < B khi nào? (Gợi ý: ý nghĩa của cờ C khi trừ ở Câu 1.6b.) Kiểm tra điều kiện của Sinh viên trên hai dòng 0xC8 và 0x0A của Bảng 7 bằng cách tính A − B trên 9 bit. Sau đó, so sánh chi phí: bộ so sánh trực tiếp hai số 8 bit cần bảng chân trị bao nhiêu hàng, so với "ALU + LT"? Khi nào vẫn nên dùng bộ so sánh riêng?

Với số không dấu, A<B khi phép trừ A-B có mượn, tức C=0 (C=1 khi A>=B).
Trên 9 bit: 0x0C8-0x00A=-0x0BE = -190, dạng 9 bit bù 2 là 0x142.
Bộ so sánh trực tiếp 8 bit có 2^16=65.536 cặp đầu vào.
ALU+LT tái sử dụng bộ trừ và chỉ thêm logic nhỏ. Bộ so sánh riêng vẫn hữu ích khi cần kết quả song song hay độ trễ thấp, ít điện năng trong thiết kế chuyên biệt, hoặc ALU đang bận hoặc không có sẵn.

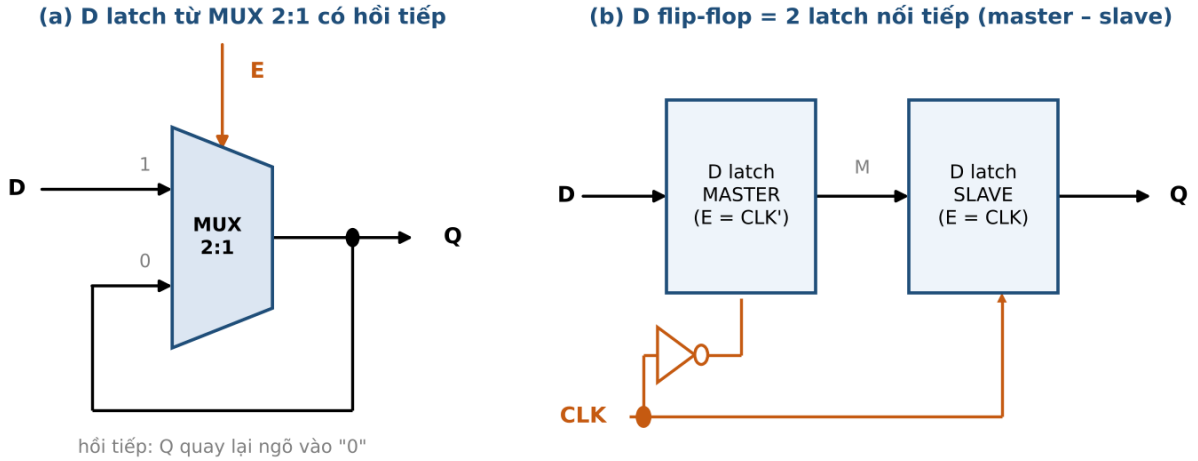


THÍ NGHIỆM 3 — Mạch tuần tự: D latch và D flip-flop

Thời lượng gợi ý: 45 phút · Circuit: **TN3 (mạch con: DLATCH, DFF_MS)** · Điểm: 2,5

Tình huống

ALU ở Thí nghiệm 2 là mạch tổ hợp: ngõ ra đổi ngay khi ngõ vào đổi và **không nhớ** gì. Muốn CPU giữ lại kết quả để dùng cho lệnh sau, cần **thiết bị lưu trữ** (slide Chương 4). Sinh viên sẽ tự dựng D latch từ bảng trạng thái ở slide, so sánh nó với D flip-flop có sẵn, rồi ghép hai latch thành một flip-flop và tìm hiểu vì sao thanh ghi phải dùng flip-flop.



Hình 2. *(a) D latch dựng từ MUX 2:1 có hồi tiếp; (b) D flip-flop ghép từ hai latch (master – slave).*

A. D latch dựng từ bảng trạng thái

Slide Chương 4 mô tả D latch bằng bảng trạng thái kế tiếp: **Q+ = Q khi E = 0** (giữ), **Q+ = D khi E = 1** (nhận). Đó chính là một **MUX 2:1** mà ngõ vào "giữ" được nối với **ngõ ra Q của chính nó** (Hình 2a).

DỰNG MẠCH **Bước 3.1.** Trong circuit **DLATCH**: Input Pin **D, E**; Output Pin **Q**; một Multiplexer 1 bit (Select = E, ngõ vào 1 = D, ngõ vào 0 = **Q nối ngược về**).

DỰ ĐOÁN Chọn **Simulate ▸ Reset Simulation**, rồi thực hiện lần lượt các thao tác ở Bảng 9 (đổi E và D bằng Poke). Điền Q dự đoán **trước**; nếu Q chưa xác định thì ghi **x**.

QUAN SÁT Sau đó điền cột "Q đo".

Bảng 9. Chuỗi thao tác trên DLATCH

Bước	E	D	Q dự đoán	Q đo
1	0	0	x	x
2	0	1	x	x
3	1	1	1	1
4	0	1	1	1
5	0	0	1	1
6	1	0	0	0
7	1	1	1	1
8	1	0	0	0
9	0	0	0	0

Bước	E	D	Q dự đoán	Q đo
10	0	1	0	0

Câu 3.1 (Hiểu). Vì sao ở bước 1–2 Q chưa xác định dù đã có D? Điều đó nói gì về việc phải **khởi tạo** (reset) thiết bị lưu trữ trong mạch thật?

Khi E=0, latch giữ Q cũ qua hồi tiếp. Ban đầu Q chưa được khởi tạo nên hai bước đầu vẫn là x dù D đã có giá trị. Mạch thật cần reset để trạng thái sau bật nguồn xác định trước khi sử dụng.

Câu 3.2 (Phân tích). Slide Chương 4 định nghĩa mạch tổ hợp là mạch **không có hồi tiếp**; DLATCH có đường hồi tiếp từ Q về MUX. Khi E = 0, mô tả đường đi của tín hiệu để giải thích vì sao hồi tiếp làm mạch "nhớ". Khi E = 1 thì hồi tiếp còn tác dụng không?

Khi E=0, MUX chọn ngõ Q hồi tiếp, nên Q được đưa lại chính ngõ vào và giữ nguyên dù D thay đổi.
Khi E=1, MUX chọn D. Hồi tiếp không quyết định giá trị nữa, latch trong suốt và Q theo D.

B. Latch và flip-flop: cùng ngõ vào, khác kết quả

DỰNG MẠCH **Bước 3.2.** Trong circuit **TN3**: Input Pin **D** và **CLK** (đổi bằng Poke, **không** dùng Clock tự chạy); một DLATCH với E = CLK (ngõ ra **Q1**); một **D Flip-Flop** (Memory, kích cạnh lên – Rising Edge) với ngõ vào D, CLK (ngõ ra **Q2**).

DỰ ĐOÁN Bảng 10 cho CLK và D theo từng **khe thời gian** (mỗi cột một khe; tại ranh giới giữa hai khe chỉ đổi **một** tín hiệu: CLK hoặc D, không bao giờ cả hai). Giả sử ban đầu Q1 = Q2 = 0. Điền hàng dự đoán Q1, Q2. Đây là bài "hoàn thành dạng sóng" ở slide Chương 4.

QUAN SÁT Đưa mạch về Q1 = Q2 = 0 (Reset Simulation; D = 0; CLK 0 → 1 → 0). Sau đó Poke từng khe theo bảng và ghi Q1, Q2 đo được. Hàng Q3 để trống đến Bước 3.3.

Bảng 10. Dạng sóng theo khe thời gian

Khe	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CLK	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
D	0	1	1	0	0	1	1	1	1	0	0	1	1	0	0	0
Q1 dự đoán	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q1 đo	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q2 dự đoán	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q2 đo	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q3 đo (DFF_MS)	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0

Câu 3.3 (Phân tích). Liệt kê các khe mà Q1 ≠ Q2. Với từng khe, giải thích bằng định nghĩa "tích cực theo **mức**" (latch) và "tích cực theo **cạnh**" (flip-flop) ở slide. Trong khoảng CLK = 1, D đổi một lần và Q1 đổi theo, còn Q2 thì không: vì sao?

Q1 # Q2 ở các khe 3, 4, 5, 11, 12, 13.
Latch theo mức: khi CLK=1, Q1 theo D nên các khe 3 và 11 Q1 đổi ngay.
Flip-flop chỉ chốt tại cạnh lên nên giữ giá trị đã lấy mẫu (Q2). Thay đổi D giữa hai cạnh không làm Q2 đổi.

C. Mở rộng: Ghép hai latch thành flip-flop

Hình 2b: **master** có E = CLK' (tức là trong suốt khi CLK = 0), **slave** có E = CLK (trong suốt khi CLK = 1).

DỰ ĐOÁN **Trước khi dựng**, dự đoán: Q của mạch này đổi giá trị vào lúc nào? Chọn một và nêu lý do.

☐ Suốt lúc CLK = 1	☐ Suốt lúc CLK = 0	☑ Ngay tại cạnh lên	☐ Ngay tại cạnh xuống

Khi CLK = 0: Mạch Master có E = CLK' = 1 nên ở trạng thái trong suốt (cho phép nhận và cập nhật dữ liệu từ ngõ vào D), trong khi mạch Slave có E = CLK = 0 nên bị giữ nguyên giá trị cũ. Khi có sự chuyển mức từ 0 lên 1, mạch Master khóa lại để chốt giá trị dữ liệu vừa thu nhận, đồng thời mạch Slave chuyển sang E = 1, cho phép truyền dữ liệu từ Master ra ngõ ra Q.

DỰNG MẠCH **Bước 3.3.** Circuit **DFF_MS**: Input Pin D, CLK; Output Pin Q; hai mạch con DLATCH và một cổng NOT. Đưa DFF_MS vào circuit TN3 (dùng chung D, CLK), thêm Output Pin **Q3** và điền hàng "Q3 đo" của Bảng 10.

Câu 3.4 (Phân tích). Điền Bảng 11 cho hai pha của mạch master–slave, rồi so sánh hàng Q3 với hàng Q2 ở Bảng 10. Chúng có giống nhau không? Nếu khác, ở khe nào và vì sao?

Bảng 11. Hai pha của master–slave

Pha	Master (trong suốt / giữ)	Slave (trong suốt / giữ)	Q có thể đổi không?
CLK = 0	Trong suốt	Giữ	Không
CLK = 1	Giữ	Trong suốt	Có thể đổi theo giá trị master đã chốt

Q2 và Q3 giống nhau, đều cấu tạo từ 2 latch (master, slave)

D. Vì sao thanh ghi dùng flip-flop mà không dùng latch?

DỰNG MẠCH **Bước 3.4.** Trong circuit TN3, dựng hai chuỗi hai tầng: **(i)** hai DLATCH nối tiếp (Q của latch 1 vào D của latch 2), cả hai có E = CLK; **(ii)** hai D Flip-Flop nối tiếp cùng CLK. Cả hai chuỗi dùng chung ngõ vào D.

DỰ ĐOÁN Ban đầu mọi Q = 0. Mỗi **xung** là CLK: 0 → 1 → 0, D giữ nguyên trong xung. Điền Q1, Q2 (viết dạng "Q1 Q2") sau mỗi xung, rồi đo.

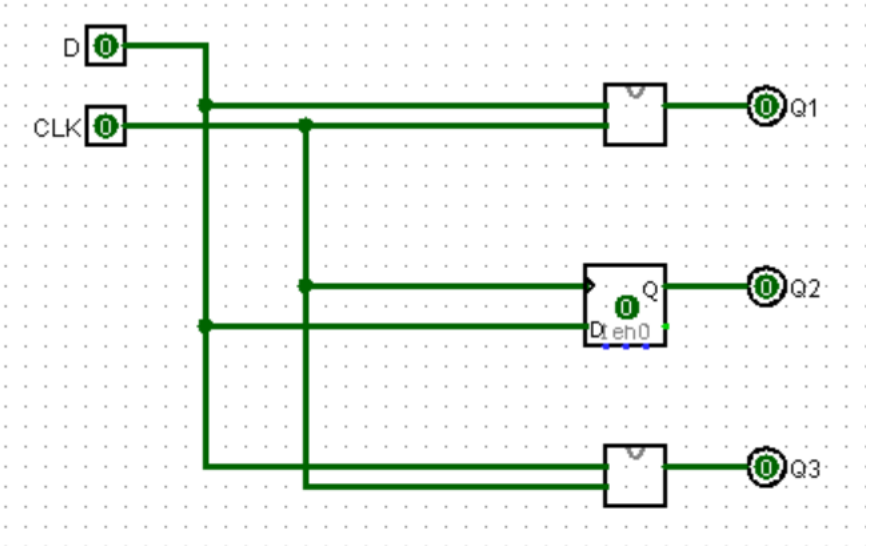
Bảng 12. Chuỗi hai tầng: latch và flip-flop

Xung	D	2 latch: dự đoán	2 latch: đo	2 flip-flop: dự đoán	2 flip-flop: đo
1	1	11	11	10	10
2	0	00	00	01	01
3	0	00	00	00	00

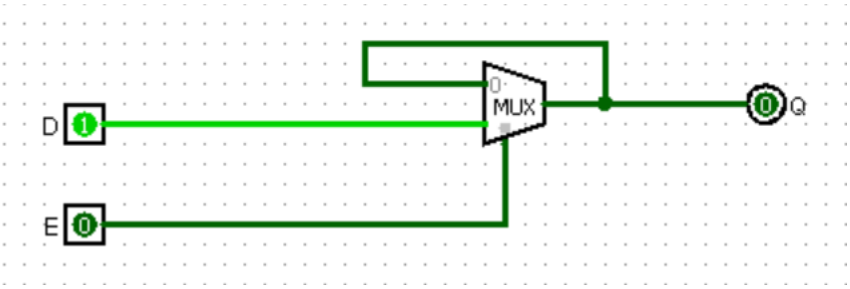
Câu 3.5 (Đánh giá). Từ Bảng 12, dữ liệu đi qua mấy tầng trong **một** xung ở mỗi cấu hình? Từ đó dự đoán điều gì xảy ra với lệnh **R1 ← R1 + R0** (ngõ ra ALU quay lại ngõ vào chính thanh ghi R1) khi R1 được làm bằng latch trong suốt lúc CLK = 1, và khi R1 làm bằng flip-flop. (Sinh viên sẽ kiểm chứng ở Thí nghiệm 4.)

Hai latch cùng trong suốt mức cao cho dữ liệu đi qua cả hai tầng trong một pha CLK=1 (race-through).
Hai flip-flop cùng cạnh chỉ lấy mẫu đồng thời: dữ liệu mới tiến tối đa một tầng mỗi xung. Vì vậy thanh ghi dùng flip-flop để tạo ranh giới chu kỳ xác định. Nếu R1 là latch trong suốt khi CLK=1, tổng R1+R0 quay về đầu vào khi latch còn mở và có thể được cộng lặp/race-through, nên kết quả phụ thuộc trễ và không ổn định; với flip-flop, R1 chỉ tăng một lần mỗi cạnh lên.

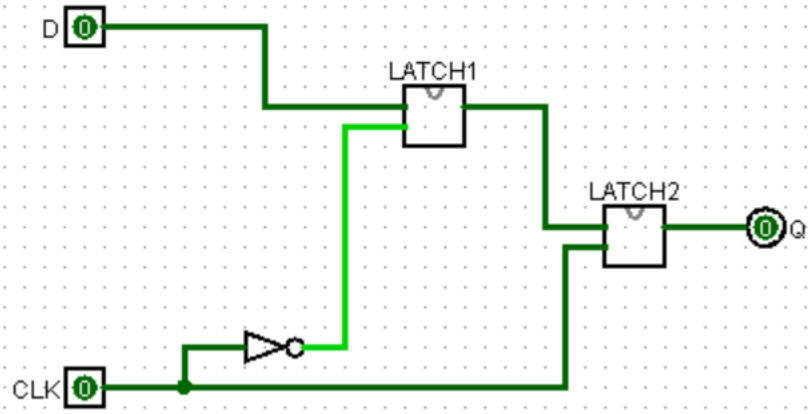
TN3 :



DLATCH :



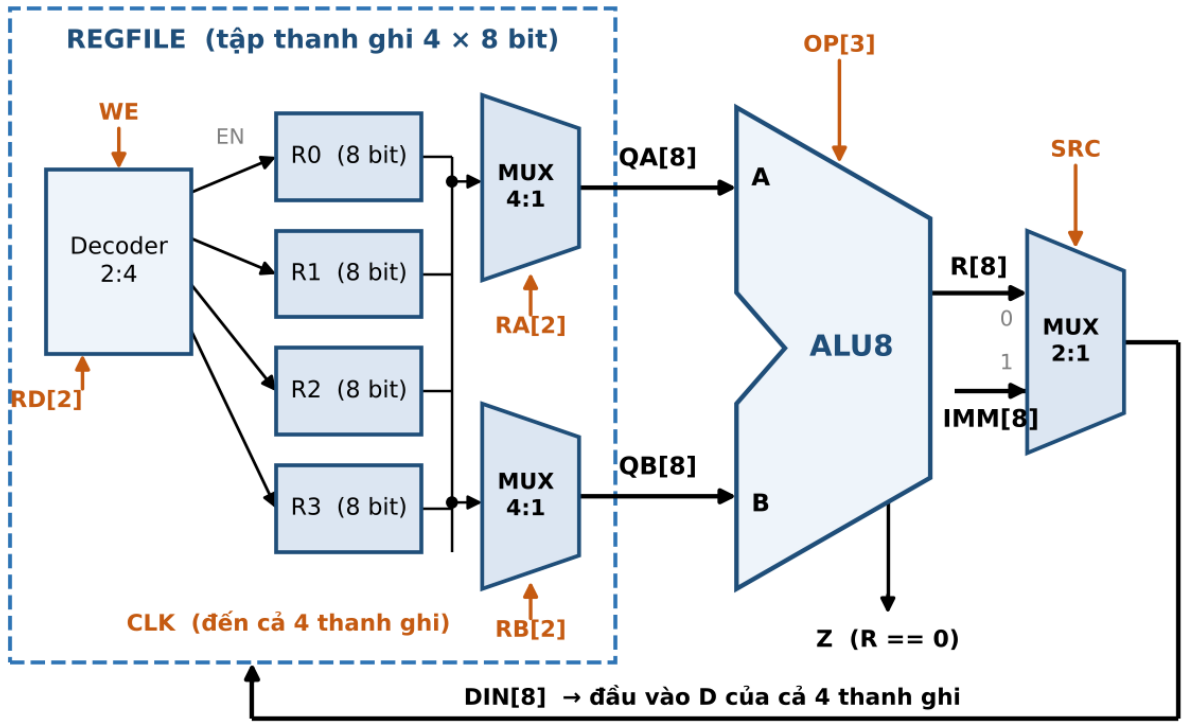
DFF_MS :



THÍ NGHIỆM 4 — Tập thanh ghi và đường dữ liệu 8 bit

Thời lượng gợi ý: 60 phút · Circuit: **REGFILE, DATAPATH** · Điểm: 3,0 (+1,0 mở rộng)

> **Tình huống**
> Slide Chương 5 minh hoạ lệnh **C = A + B**: đưa A, B từ bộ nhớ vào **tập thanh ghi**, **ALU** tính, kết quả ghi lại vào tập thanh ghi. Sinh viên ghép các khối đã dựng thành **đường dữ liệu** của một CPU thu nhỏ và điều khiển nó bằng tay: mỗi "lệnh" là một bộ tín hiệu điều khiển cộng với một xung CLK.



Hình 3. *Đường dữ liệu mini: REGFILE (decoder 2:4 + 4 thanh ghi + 2 MUX đọc) → ALU8 → MUX chọn nguồn ghi (SRC).*

A. Tập thanh ghi 4 × 8 bit

Slide Chương 5 vẽ tập thanh ghi với một cổng đọc. ALU cần **hai toán hạng cùng lúc** nên REGFILE ở đây có **hai cổng đọc** (RA, RB) và **một cổng ghi** (RD).

DỰNG MẠCH **Bước 4.1.** Trong circuit **REGFILE**: Input Pin **CLK, WE** (1 bit), **RD** (2 bit, địa chỉ ghi), **DIN** (8 bit), **RA, RB** (2 bit, địa chỉ đọc); Output Pin **QA, QB** (8 bit) và **R0 … R3** (8 bit, để theo dõi). Dùng: một **Decoder 2:4** (Select = RD; nối WE vào ngõ Enable nếu có, hoặc dùng 4 cổng

AND với WE), **4 Register** 8 bit (D = DIN, En = ngõ ra thứ k của decoder, Clock = CLK) và **2 Multiplexer 4:1** 8 bit chọn Rk theo RA và theo RB.

DỰ ĐOÁN Với các bước 5–8 của Bảng 13, điền cột "R1 dự đoán" **trước** khi Poke. Mỗi bước kết thúc bằng một xung CLK (0 → 1 → 0) trừ khi ghi chú khác.

QUAN SÁT Ghi R0 … R3 (hex) sau từng bước.

Bảng 13. Ghi và đọc trên REGFILE

Bước	Thao tác	R1 dự đoán	R0	R1	R2	R3
0	Vừa Reset Simulation (WE = 0)	x	x	x	x	x
1	RD = 0, WE = 1, DIN = 0x11, xung CLK	–	0x11	x	x	x
2	RD = 1, WE = 1, DIN = 0x22, xung CLK	–	0x11	0x22	x	x
3	RD = 2, WE = 1, DIN = 0x33, xung CLK	–	0x11	0x22	0x33	x
4	RD = 3, WE = 1, DIN = 0x44, xung CLK	–	0x11	0x22	0x33	0x44
5	RD = 1, **WE = 0**, DIN = 0x99, xung CLK	0x22	0x11	0x22	0x33	0x44
6	RD = 1, WE = 1, DIN = 0x99, **CLK vẫn = 0**	0x22	0x11	0x22	0x33	0x44
7	CLK: 0 → 1 (giữ CLK = 1)	0x99	0x11	0x99	0x33	0x44
8	CLK: 1 → 0	0x99	0x11	0x99	0x33	0x44

Câu 4.1 (Hiểu). Vì sao cần tín hiệu WE ngoài CLK? Điều gì xảy ra nếu bỏ WE (mỗi xung CLK đều ghi DIN vào thanh ghi do RD chọn) khi ta chỉ muốn **so sánh** hai thanh ghi mà không muốn ghi kết quả?

WE bật/tắt thao tác ghi độc lập với xung clock. Khi chỉ đọc/so sánh, WE=0 giữ nguyên toàn bộ thanh ghi; nếu bỏ WE, cạnh clock sẽ ghi DIN vào thanh ghi RD và có thể phá hỏng dữ liệu dù lệnh không yêu cầu ghi.

Câu 4.2 (Phân tích). Khi RD = RA = 2, WE = 1 và DIN thay đổi: QA đổi **ngay khi DIN đổi** hay **chỉ sau cạnh lên**? Vì sao điều này quan trọng cho lệnh R2 ← R2 + R1 (liên hệ Bảng 12)?

QA là ngõ ra của R2 qua MUX đọc; DIN thay đổi không làm QA đổi ngay. QA chỉ đổi sau cạnh lên khi R2 được ghi. Điều này tạo ranh giới trạng thái cho R2<-R2+R1: ALU tính từ giá trị cũ, rồi thanh ghi chốt một kết quả ở cạnh, tránh dữ liệu chạy xuyên vòng hồi tiếp như hai latch cùng trong suốt (Bảng 12).

B. Ghép đường dữ liệu

DỰNG MẠCH **Bước 4.2.** Trong circuit **DATAPATH**: dùng mạch con REGFILE, mạch con ALU8 và một Multiplexer 2:1 (8 bit) chọn DIN: **SRC = 0 → R** (kết quả ALU), **SRC = 1 → IMM** (hằng

số). Input Pin: **IMM** (8 bit), **SRC**, **OP** (3 bit), **RA, RB, RD** (2 bit), **WE**, **CLK**; Output Pin: **R0 … R3**, **R**, **Z**. Nối theo Hình 3 (A ← QA, B ← QB).

> **Quy ước "một lệnh"**
> - Một lệnh = đặt đúng các tín hiệu điều khiển (SRC, OP, RA, RB, RD, WE, IMM) **rồi** tạo một xung CLK (0 → 1 → 0).
> - Lệnh **nạp hằng**: SRC = 1. Lệnh **ALU**: SRC = 0. Lệnh **chỉ kiểm tra** (không ghi): WE = 0, đọc cờ Z.
> - Ô không dùng đến ghi dấu "–".

DỰ ĐOÁN Điền **từ vựng điều khiển** của 5 lệnh ở Bảng 14 (số thanh ghi viết dạng thập phân, OP dạng nhị phân, IMM dạng hex).

Bảng 14. Tín hiệu điều khiển của từng lệnh

Lệnh	SRC	OP	RA	RB	RD	WE	IMM
(1) R0 ← 0x05	1	-	-	-	0	1	0x05
(2) R1 ← 0x07	1	-	-	-	1	1	0x07
(3) R2 ← R0 + R1 (C = A + B ở slide)	0	000	0	1	2	1	-
(4) R3 ← R2 − R0	0	010	2	0	3	1	-
(5) So sánh R3 với R1 (không ghi)	0	010	3	1	-	0	-

QUAN SÁT Chạy lần lượt 5 lệnh (từ trạng thái Reset Simulation) và ghi R0 … R3 (hex) và cờ Z sau mỗi lệnh.

Bảng 15. Kết quả chạy chương trình (1)–(5)

Sau lệnh	R0	R1	R2	R3	Z
(1)	0x05	x	x	x	–
(2)	0x05	0x07	x	x	–
(3)	0x05	0x07	0x0C	x	0
(4)	0x05	0x07	0x0C	0x07	0
(5)	0x05	0x07	0x0C	0x07	1

Câu 4.3 (Hiểu). Lệnh (5) không ghi gì nhưng vẫn hữu ích. Z = 1 cho biết điều gì? So với quy tắc "MSB" ở Thí nghiệm 2, kết luận "bằng nhau" từ Z có luôn đúng không? Vì sao?

Z=1 nghĩa là kết quả R của ALU bằng 0. Ở lệnh trừ R3-R1, điều đó cho biết R3=R1. Phép trừ bằng 0 luôn nhận ra hai toán hạng 8 bit bằng nhau, không phụ thuộc cách đọc có dấu/không dấu, với phép ALU khác, Z chỉ nói kết quả phép đó bằng 0.

C. Chương trình ngắn: hoán đổi hai thanh ghi không cần thanh ghi tạm

Hoán đổi R0 và R1 chỉ bằng **ba lệnh XOR** (phép toán 111 của ALU): **(1)** R0 ← R0 ⊕ R1; **(2)** R1 ← R0 ⊕ R1; **(3)** R0 ← R0 ⊕ R1.

DỰ ĐOÁN Cho R0 = 0x05, R1 = 0x07 (nạp bằng lệnh nạp hằng nếu cần). Tính tay (XOR từng bit) và điền cột "dự đoán" của Bảng 16 **trước khi** chạy.

QUAN SÁT Chạy 3 lệnh trên DATAPATH và điền cột "đo".

Bảng 16. Hoán đổi bằng XOR

Sau lệnh	R0 dự đoán	R1 dự đoán	R0 đo	R1 đo
Ban đầu	0x05	0x07	0x05	0x07
(1) R0 ← R0 ⊕ R1	0x02	0x07	0x02	0x07
(2) R1 ← R0 ⊕ R1	0x02	0x05	0x02	0x05
(3) R0 ← R0 ⊕ R1	0x07	0x05	0x07	0x05

Câu 4.4 (Phân tích). Đặt a, b là giá trị ban đầu của R0, R1. Dùng các tính chất của XOR (x ⊕ x = 0, x ⊕ 0 = x, giao hoán, kết hợp) viết giá trị của R0, R1 sau mỗi lệnh theo a, b để chứng minh phương pháp đúng với **mọi** a, b.

Ban đầu R0=a, R1=b. (1) R0=a⊕b, R1=b. (2) R0=a⊕b, R1=(a⊕b)⊕b=a. (3) R0=(a⊕b)⊕a=b, R1=a. Hai giá trị được hoán đổi với mọi a,b nhờ x⊕x=0 và x⊕0=x.

Câu 4.5 (Đánh giá). Thử lại với hai lệnh cùng chỉ **một** thanh ghi (đặt RA = RB = RD = 0 ở lệnh (1)). Kết quả là gì và vì sao? Có nên dùng thủ thuật này trong chương trình thực tế không?

Nếu RA=RB=RD=0 ở lệnh đầu, phép tính là R0⊕R0=0 và kết quả ghi đè R0 thành 0; giá trị ban đầu đã mất nên chuỗi không hoán đổi được. Không nên dùng thủ thuật XOR-swap nếu hai biến/thanh ghi có thể trùng; phép đổi chỗ tạm thông thường an toàn, dễ đọc hơn.

D. Mở rộng: Fibonacci, tràn số và vòng hồi tiếp

Dãy Fibonacci chạy trên DATAPATH với R0 = 0, R1 = 1 và vòng lặp: **R2 ← R0 + R1; R0 ← R1; R1 ← R2.** ALU **không có** phép MOV nên dùng cách đã tìm ở Câu 2.2 (R0 ← R1 | R1: đặt RA = RB = 1, OP = 101).

DỰ ĐOÁN R2 sau vòng lặp thứ k bằng F(k + 1) (F(2) = 1, F(3) = 2, …). Không cần chạy 14 vòng: tính tay F rồi điền Bảng 17 cho vòng 11–14. "C" là cờ nhớ của phép cộng R0 + R1 trong vòng đó.

Bảng 17. Fibonacci trong 8 bit

Vòng	R2 đúng (số học)	R2 lưu được (thập phân)	C	R2 có đáng tin không?
11	144	144	0	Có nếu đọc không dấu, có dấu thì ngoài miền 0–127.
12	233	233	0	Có nếu đọc không dấu, có dấu thì ngoài miền 0–127.
13	377	121	1	Không: tràn 8 bit không dấu; giá trị đã bị modulo 256.
14	610	98	1	Không: tràn 8 bit không dấu; giá trị đã bị modulo 256.

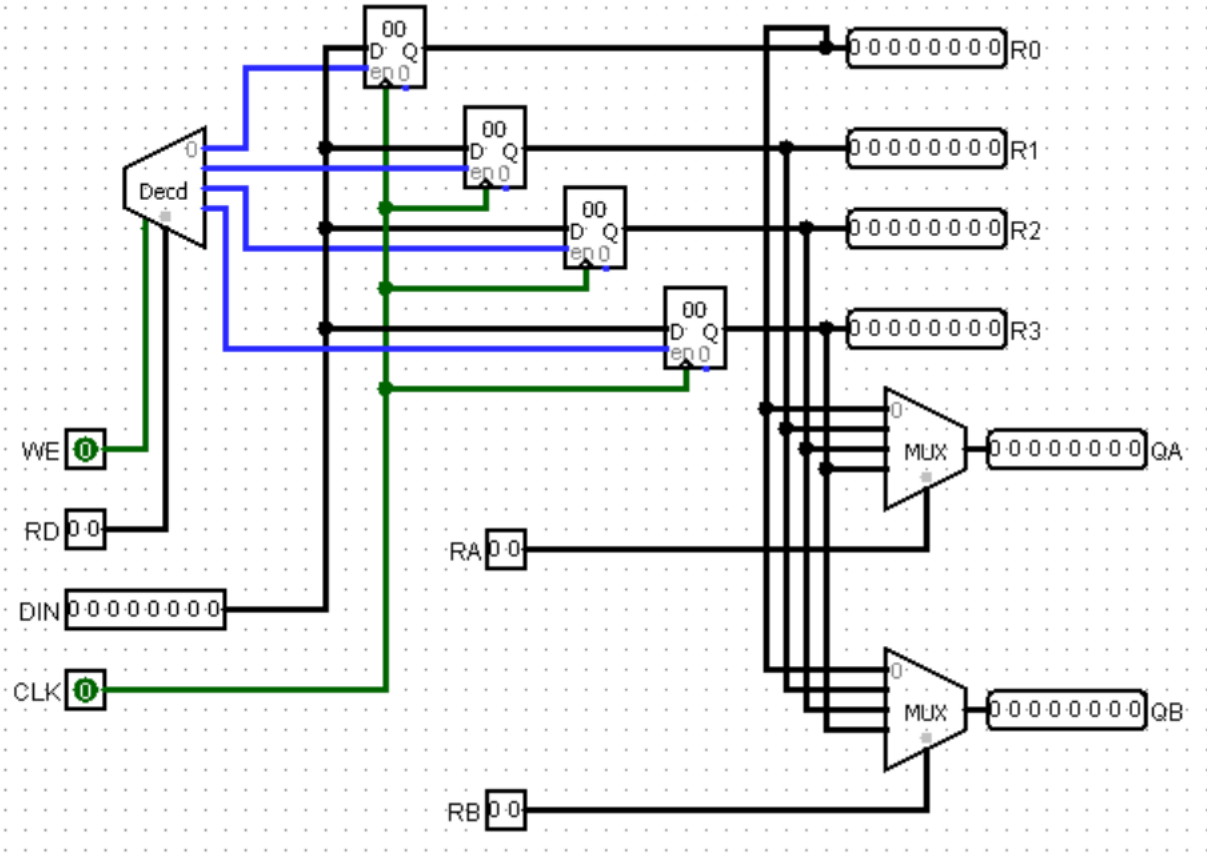
KIỂM CHỨNG Nạp R0 = 144 (0x90), R1 = 233 (0xE9) bằng lệnh nạp hằng, rồi chạy **một** vòng (3 lệnh). Ghi giá trị cuối.

R2 (hex)	0x79	R0 (hex)	0xE9	R1 (hex)	0x79

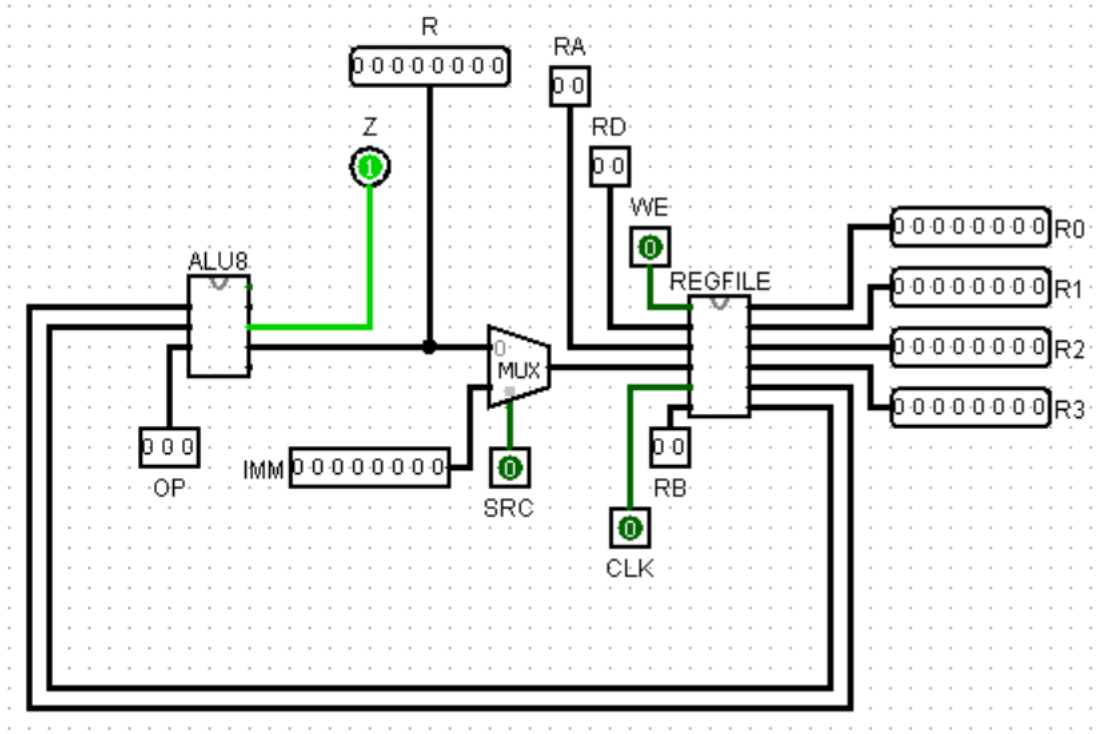
Câu 4.6 (Đánh giá). Chạy lệnh **R1 ← R1 + R0** (RA = RD = 1, RB = 0) với R0 = 3, R1 = 5: mỗi xung R1 tăng bao nhiêu? Nếu thay Register bằng một thanh ghi làm từ **latch trong suốt khi CLK = 1**, điều gì xảy ra trong lúc CLK = 1? Giải thích bằng Bảng 12 (nếu phiên bản Logisim cho phép đổi Trigger của Register sang mức cao, hãy thử và ghi lại quan sát).

Với flip-flop, mỗi cạnh lên thực hiện đúng một phép cộng: R1=5->8->11->14, tăng 3 mỗi xung. Nếu dùng latch trong suốt khi CLK=1, kết quả ALU hồi tiếp vào R1 trong lúc latch đang mở và phép cộng có thể lặp nhiều lần/race-through trong cùng pha; kết quả phụ thuộc trễ, không còn đảm bảo chỉ tăng một lần. Đây là lý do cần flip-flop ở thanh ghi.

REGFILE :



DATAPATH :



TỔNG KẾT VÀ NỘP BÀI

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File **IT012_Lab02_<MSSV>.circ** có đủ các circuit **FA, ADDSUB4, ALU8, LT, DLATCH, DFF_MS, TN3, REGFILE, DATAPATH**; mọi Input/Output Pin đều có nhãn.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp mạch vào các khung nét đứt.
☑	Số liệu trong báo cáo khớp với mạch nộp; sau khi sửa mạch con (FA, ALU8…) đã kiểm tra lại các mạch dùng nó.
☑	Đặt tên file báo cáo: **IT012_Lab02_<MSSV>_BaoCao**; nộp cả hai file theo hướng dẫn của giảng viên.

Phản tư

Câu R1. Từ Thí nghiệm 1 và 2, Sinh viên rút ra điều gì về sự khác nhau giữa "kết quả **đúng về mặt bit**" và "kết quả **đúng về mặt ý nghĩa**" (có dấu / không dấu)? Phần cứng giúp phần mềm phân biệt hai điều đó bằng cách nào?

Kết quả bit không tự mang ý nghĩa có dấu hay không dấu: cùng mẫu có thể biểu diễn hai số khác nhau. C báo carry/no-borrow cho phép toán không dấu.
V báo tràn có dấu, Z/N mô tả kết quả bit. Phần mềm chọn cách hiểu và đọc đúng cờ để xác định kết quả có biểu diễn được hay không.

Câu R2. Từ Thí nghiệm 3 và 4: trong DATAPATH, khối nào là mạch **tổ hợp**, khối nào là mạch **tuần tự**? Chỉ ra một vòng hồi tiếp (từ ngõ ra thanh ghi, qua ALU, quay về ngõ vào thanh ghi) và giải thích vì sao vòng này không gây hỗn loạn.

REGFILE (các thanh ghi) là mạch tuần tự, ALU, MUX/decoder tổ hợp và mạch LT là mạch tổ hợp.
Ví dụ R1->cổng đọc->ALU (R1+R0)->DIN->R1 là vòng hồi tiếp. Flip-flop chỉ lấy mẫu DIN tại cạnh lên, còn ALU ổn định giá trị tổ hợp trước cạnh. Kết quả được lưu cho chu kỳ sau nên vòng không tự chạy liên tục trong một chu kỳ.

Câu R3. Nêu **một** lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Ban đầu, em nhầm MSB của A-B là kết luận so sánh có dấu trong mọi trường hợp.
Em phát hiện khi thử 0x64-0x9C: hiệu bị tràn nên MSB cho kết luận sai.
Em nghĩ bài học rút ra đối với em là cần phân biệt C với V và kiểm tra cờ phù hợp với kiểu số.

Câu R4 (mở, không chấm). Để DATAPATH chạy được một chương trình có vòng lặp hay rẽ nhánh ("nếu Z = 1 thì làm A, ngược lại làm B") mà **không cần người đặt tín hiệu bằng tay**, Sinh viên cần thêm những khối nào? Ghi ý tưởng, chưa cần vẽ mạch.

Cần thêm PC, bộ nhớ lệnh, bộ giải mã/khối điều khiển tạo SRC/OP/RA/RB/RD/WE, MUX chọn PC+1 hay địa chỉ nhánh, bộ cộng tăng PC/địa chỉ nhánh và logic điều kiện dùng Z. Clock/reset đồng bộ toàn hệ thống.

Tài liệu tham khảo

- Slide bài giảng Chương 4 – Mạch số và Chương 5 – Ứng dụng mạch số (IT012, Khoa Kỹ thuật Máy tính, UIT).

- Slide bài giảng Chương 2 – Biểu diễn thông tin trong máy tính (số có dấu bù 2) và Chương 3 – Đại số Boole (bìa Karnaugh, chi phí C(B)).
- Tài liệu *Hướng dẫn sử dụng Logisim* do giảng viên cung cấp.